

Devoir surveillé terminal

Durée 2h

Nous souhaitons développer un système permettant d'exécuter des fonctions à distance par des processus quelconques. Un programme, appelé l'**exécutateur**, est en mesure de recevoir des requêtes distantes qui correspondent à une demande d'exécution d'une fonction : le nom de la fonction ainsi que la valeur de ses paramètres. La fonction est alors exécutée puis le résultat est envoyé à l'émetteur de la requête.

Les **processus locaux** qui désirent réaliser une exécution distante, passent par l'intermédiaire d'un **proxy local**. Celui-ci a pour but de mettre en contact ces processus avec un ou plusieurs exécutateurs distants. Pour cela, le proxy local établit une connexion vers chaque exécutateur connu : pour chaque connexion, un fils dédié est créé. Chaque fils enregistre toutes les fonctions disponibles, récupérées auprès de l'exécutateur, dans un segment de mémoire partagée.

Les processus locaux s'attachent au segment de mémoire partagée pour obtenir les noms des fonctions exécutables et les paramètres nécessaires, ainsi que le PID du fils du proxy correspondant. Lorsqu'ils souhaitent effectuer une exécution d'une fonction distante, les processus locaux envoient leur requête dans une file de messages. Chaque fils du proxy peut alors recevoir les requêtes qui lui sont destinées. Les résultats reçus sont envoyés dans la même file de messages.

Pour simplifier, nous caractérisons une fonction par son nom (maximum 20 caractères), le nombre de ses paramètres, la taille en octets de chacun et la taille en octets du résultat. Le type des paramètres est supposé connu par le processus qui souhaite exécuter la fonction.

1°) **Enregistrement et attachement des exécutateurs** (5,5 points). Au démarrage du proxy local, celui-ci lit depuis un fichier binaire les adresses de chaque exécutateur (adresse IPv4 + numéro de port). Une application, nommée **reg**, permet à l'utilisateur d'enregistrer les adresses de nouveaux exécutateurs dans ce fichier. L'application prend en arguments le nom du fichier binaire, l'adresse IPv4 et le numéro de port du nouvel exécutateur. Avant de l'ajouter, elle vérifie que l'adresse n'est pas déjà présente dans le fichier.

a) Donnez une structure en C, nommée **adresse_t**, permettant de représenter l'adresse d'un exécutateur. Expliquez votre choix et donnez la taille en mémoire de cette structure.

b) Expliquez en quelques phrases comment vérifier qu'une adresse n'est pas déjà présente dans le fichier.

c) Nous supposons l'existence de la fonction `int est_presente(int fd, adresse_t adr)` qui retourne 1 si l'adresse existe dans le fichier, 0 sinon. Le paramètre **fd** correspond au descripteur du fichier binaire ouvert en lecture/écriture et le paramètre **adr** l'adresse que l'on souhaite vérifier. Écrivez la procédure `void ajouter_adresse(char *nomFichier, adresse_t adr)` qui permet d'ajouter la nouvelle adresse à la fin du fichier si elle n'est pas déjà présente. Si le fichier n'existe pas, il est créé. Le paramètre **nomFichier** est le nom du fichier binaire et le paramètre **adr** la nouvelle adresse.

d) Nous supposons l'existence de la procédure `void fils(adresse_t adr)` qui correspond à la routine principale de chaque processus fils du proxy local. Elle prend en paramètre l'adresse d'un exécutateur et établit une connexion TCP avec celui-ci. Elle récupère ensuite toutes les fonctions de l'exécutateur qu'elle enregistre dans le segment de mémoire partagée. Puis elle se met en attente de requêtes de la part de processus locaux depuis la file de messages. Écrivez la fonction `void creation_fils(char *nomFichier)` qui prend en paramètre le nom du fichier contenant les adresses des exécutateurs et qui crée un fils pour chacune. Chaque fils exécute la procédure **fils**.

e) Nous supposons que le programme principal du proxy prend en argument le nom du fichier contenant les adresses des exécutateurs. Il crée les fils (avec la fonction **creation_fils**) puis attend que l'utilisateur presse CTRL+C. Donnez le code du **main**.

2°) Communications réseaux (4 points).

- a) Expliquez, à votre avis, pourquoi le mode connecté est préférable au mode non connecté dans cette application.
- b) Combien de sockets sont nécessaires sur chaque proxy (tous ses fils confondus)? Et sur un exécuteur (tous ses fils confondus)?
- c) Donnez tous les échanges possibles entre les fils et les exécuteurs distants. Vous spécifierez les données échangées (type, taille, rôle).
- d) Donnez la partie du code de la procédure `fils` (correspondant à un fils d'un proxy) qui établit une connexion TCP avec l'exécuteur dont l'adresse est passée en paramètre de la procédure.

3°) Segment de mémoire partagée (5 points).

- a) Le segment de mémoire partagée contient les "signatures" de chaque fonction, ajoutées par les différents fils du proxy local. Donnez une structure en C, nommée `fonction_t`, permettant de représenter une signature de fonction en mémoire.
- b) Expliquez comment stocker cette structure dans le segment de mémoire partagée.
- c) Proposez une solution permettant d'ajouter des fonctions dans le segment. Elle devra permettre aussi de parcourir les fonctions présentes dans le segment.
- d) Donnez le code de la fonction `void* attacher_segment(key_t cle)` permettant d'attacher le segment de mémoire partagée en mémoire. Le paramètre `cle` est la clé IPC du segment. Si le segment n'existe pas, la fonction retourne `NULL`.
- e) Écrivez la fonction `void ajouter_fonction(fonction_t fct, void *position)` qui permet à un fils du proxy d'ajouter la fonction à la position spécifiée dans le segment de mémoire partagée. Le paramètre `fct` correspond à la signature de la fonction, le paramètre `position` correspond à la position à laquelle ajouter la fonction (il s'agit bien de la position où ajouter la fonction et non pas de l'adresse du segment de mémoire partagée).

4°) File de messages (2 points).

- a) Proposez une solution permettant aux processus locaux d'envoyer leurs requêtes dans la file de messages, spécifiquement à un fils du proxy et de pouvoir lire les réponses.
- b) Proposez la structure `resultat_t` correspondant au message envoyé par un fils du proxy à un processus local. Précisez les champs utilisés et donnez les limites éventuelles dues à votre choix.
- c) Donnez le code de la fonction `void envoyer_resultat(int msgid, resultat_t resultat)` permettant d'envoyer un résultat dans la file de messages. Le paramètre `msgid` est l'identifiant de la file de messages et `resultat` le résultat à envoyer.

5°) Concurrency (3,5 points).

- a) Expliquez tous les problèmes de concurrence qui doivent être gérés dans le système.
- b) Combien de sémaphores sont nécessaires? Expliquez le rôle de chacune et comment l'utiliser. Aucun processus ne doit être bloqué inutilement.
- c) Donnez le code permettant de créer le tableau de sémaphores nécessaire et de l'initialiser. La clé IPC correspond à la constante `CLE_SEM`. Si le tableau existe déjà, il doit être supprimé avant de créer le nouveau tableau.

Annexes : quelques en-têtes de fonctions C

```

void      _exit(int code);
unsigned int alarm(unsigned int nb_secondes);
int       atexit(void (*procedure)(void));
void      clearerr(FILE *flux);
int       close(int fd);
int       closedir(DIR *repertoire);
int       creat(const char *chemin, mode_t mode);
int       dup(int ancien_fd);
int       dup2(int ancien_fd, nouveau_fd);
int       execve(const char *fichier, char *const argv[], char *const envp[]);
void      exit(int statut);
int       fcntl(int fd, int commande, ...);
int       fclose(FILE *flux);
FILE      *fdopen(int fd, const char *mode);
int       feof(FILE *flux);
int       ferror(FILE *flux);
int       fflush(FILE *flux);
int       fileno(FILE *flux);
FILE      *fopen(const char *chemin, const char *mode);
pid_t     fork();
size_t    fread(void *tampon, size_t taille, size_t nombre_elements, FILE *flux);
FILE      *freopen(const char *chemin, const char *mode, FILE *flux);
int       fseek(FILE *flux, long offset, int point_depart);
int       fstat(int fd, struct stat *tampon);
int       ftell(FILE *flux);
size_t    fwrite(const void *tampon, size_t taille, size_t nombre_elements, FILE *flux);
pid_t     getpid();
pid_t     getppid();
int       kill(pid_t pid, int numero_signal);
off_t     lseek(int fd, off_t offset, int point_depart);
int       lstat(const char *chemin, struct stat *tampon);
void      *memcpy(void *destination, const void *source, size_t taille);
void      *memset(void *tampon, int caractere, size_t nombre_elements);
int       mkfifo(const char *nomFichier, mode_t mode);
int       open(const char *chemin, int options);
int       open(const char *chemin, int options, mode_t mode);
DIR        *opendir(const char *chemin);
int       pause(void);
int       pipe(int tube[2]);
ssize_t    read(int fd, void *tampon, size_t taille);
struct dirent *readdir(DIR *repertoire);
void      rewind(FILE *flux);
void      rewinddir(DIR *repertoire);
void      (*signal(int numero_signal, void (*handler)(int)))(int)
int       sigaction(int numero_signal, const struct sigaction *action,
                    struct sigaction *ancienne_action);
int       sigaddset(sigset_t *ensemble, int numero_signal);
int       sigdelset(sigset_t *ensemble, int numero_signal);
int       sigemptyset(sigset_t *ensemble);
int       sigfillset(sigset_t *ensemble);
int       sigismember(sigset_t *ensemble, int numero_signal);
int       sigpending(sigset_t *ensemble);
int       sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int       sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int       sigsuspend(const sigset_t *ensemble);
int       sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int       sigtimedwait(const sigset_t *ensemble, siginfo_t *info,
                       const struct timespec *timeout);
unsigned int sleep(unsigned int nombre_secondes);
int       stat(const char *chemin, struct stat *tampon);

```

```

time_t      time(time_t *temps);
int          truncate(const char *chemin, off_t offset);
int          ftruncate(int fd, off_t offset);
int          truncate64(const char *chemin, off64_t offset);
int          ftruncate64(int fd, off64_t offset);
int          unlink(const char *nomFichier);
pid_t        wait(int *statut);
pid_t        waitpid(pid_t pid, int *statut, int options);
ssize_t      write(int fd, const void *tampon, size_t taille);

```

Annexes : constantes associées à des paramètres ou des champs de structures

```

options (open, fcntl) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
mode (creat, open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH,
S_IWOTH, S_IXOTH
commande (fcntl) : F_GETFL, F_SETFL
mode (fopen, fdopen, freopen) : "r", "r+", "w", "w+", "a", "a+"
point_depart (lseek, fseek) : SEEK_SET, SEEK_CUR, SEEK_END
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP,
S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
handler (signal, champ sa_handler de struct sigaction) : SIG_IGN, SIG_DFL
sa_flags (champ sa_flags de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
options (waitpid) : WNOHANG, WUNTRACED

```

Annexes : macros

Sur `statut` de `wait`, `waitpid` : `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG`, `WIFSTOPPED`, `WSTOPSIG`
 Sur le champ `st_mode` de `struct stat` : `S_ISREG`, `S_ISDIR`, `S_ISFIFO`

Annexes : quelques structures C

```

struct stat {
    dev_t      st_dev;
    ino_t      st_ino;
    mode_t     st_mode;
    nlink_t    st_nlink;
    uid_t      st_uid;
    gid_t      st_gid;
    dev_t      st_rdev;
    off_t      st_size;
    blksize_t  st_blksize;
    blkcnt_t   st_blocks;
    time_t     st_atime;
    time_t     st_mtime;
    time_t     st_ctime;
};

struct dirent {
    char d_name[256];
};

struct sigaction {
    void (*sa_handler) (int);
    void (*sa_sigaction) (int, siginfo_t*, void*);
    sigset_t sa_mask;
    int sa_flags;
};

union sigval_t {
    int sival_int;
    void *sival_ptr;
};

struct siginfo_t {
    int si_signo;
    int si_code;
    sigval_t si_value;
    pid_t si_pid;
    ...
};

struct timespec {
    long tv_sec;
    long tv_nsec;
};

```